**UNIVERSITAS MULTI DATA PALEMBANG**
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

**TEMPLAT LAPORAN**

# TUGAS 2

## Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

| | |
|---|---|
| **Nama dan NIM** | Deisy Callisca Chandra – 2428240054 |
| **Kelas** | SI5B |
| **Judul project / topik API** | Kebun – Tanaman (RESTful API Express.js, resource /plants) |
| **Tautan repositori** | https://github.com/DeisyCC32/tugas1-restful--2428240054- |
| **Dosen pengampu** | Nur Rachmat, M.Kom. |
| **Tanggal pengumpulan** | |

# Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

## A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

## B. Bukti yang diminta

| No | Bukti | Bagian | Jenis |
|---|---|---|---|
| 1 | Daftar request dalam Collection | Bab 2 | Screenshot (wajib) |
| 2 | Cuplikan test script | Bab 3 | Kode teks (wajib) |
| 3 | Ringkasan Collection Runner | Bab 4 | Screenshot (wajib) |
| 4 | Hasil Newman di terminal | Bab 5 | Screenshot (pengayaan) |

## C. Syarat tangkapan layar

| Bukti | Yang harus terlihat |
|---|---|
| Collection | Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar. |
| Ringkasan Runner | Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal). |
| Newman | Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir. |
| Larangan | Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit. |

> **Sebelum mengambil screenshot Runner**
> Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

# Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

## 1.1 Deskripsi singkat API

**Penjelasan:**
API ini bertopik Kebun – Tanaman dengan entitas utama tanaman (plant) yang memiliki field id, namaTanaman, jenis (sayur, buah, hias), lokasiBedeng, tanggalTanam, dan sudahPanen. API dibangun dengan Express.js dan menyimpan data dalam array di memori tanpa database. API menyediakan operasi CRUD lengkap, validasi input, serta respons error berformat JSON (404 dan 400). Pengujian dilakukan di Postman terhadap server lokal http://localhost:3000.

## 1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey` atau mekanisme serupa.

| No | Metode | Alamat | API key | Status sukses | Fungsi |
|---|---|---|---|---|---|
| 1 | GET | / | Tidak | 200 | Menampilkan informasi API dan daftar endpoint |
| 2 | GET | /plants | Tidak | 200 | Mengambil seluruh data tanaman |
| 3 | GET | /plants?jenis=sayur | Tidak | 200 | Memfilter tanaman berdasarkan jenis |
| 4 | GET | /plants/:id | Tidak | 200 | Mengambil satu tanaman (404 bila id tidak ada) |
| 5 | POST | /plants | Tidak | 201 | Menambah tanaman baru (400 bila validasi gagal) |
| 6 | PUT | /plants/:id | Tidak | 200 | Mengubah seluruh data tanaman (404 bila id tidak ada) |
| 7 | DELETE | /plants/:id | Tidak | 200 | Menghapus tanaman (404 bila id tidak ada) |

## 1.3 Variabel Environment

Tuliskan variabel yang Anda simpan di Environment. Jangan menuliskan nilai rahasia sungguhan.

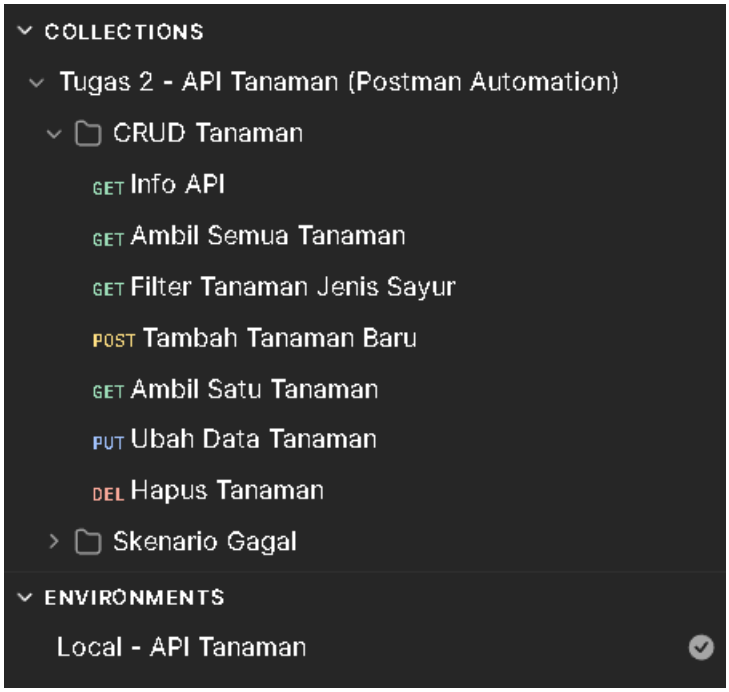
| No | Nama variabel | Nilai (contoh) | Fungsi |
|---|---|---|---|
| 1 | base_url | http://localhost:3000 | Alamat server API |
| 2 | plantId | (kosong, diisi script) | ID tanaman dari respons POST |

# Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

## 2.1 Daftar request dalam Collection

**Harus terlihat pada screenshot:** nama Collection dan seluruh request di dalam folder. Buka semua folder.




*Gambar 1. Collection beserta seluruh request*

**Penjelasan:**
Seluruh request dikelompokkan dalam satu Collection bernama "Tugas 2 - API Tanaman (Postman Automation)" yang berisi dua folder. Folder "CRUD Tanaman" memuat 7 request untuk semua endpoint dengan urutan Info API, GET semua, GET filter, POST, GET satu data, PUT, lalu DELETE. POST sengaja diletakkan sebelum GET satu data, PUT, dan DELETE karena ketiganya memakai {{plantId}} dari respons POST. Folder "Skenario Gagal" memuat 3 request pengujian 404 dan 400 yang dijalankan setelah folder CRUD, sehingga request terakhir dapat memeriksa data yang sudah dihapus.

## 2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

| No | Nama request | Metode | Alamat (pakai {{base_url}}) | Folder |
|---|---|---|---|---|
| 1 | Info API | GET | {{base_url}}/ | CRUD Tanaman |
| 2 | Ambil Semua Tanaman | GET | {{base_url}}/plants | CRUD Tanaman |
| 3 | Filter Tanaman Jenis Sayur | GET | {{base_url}}/plants?jenis=sayur | CRUD Tanaman |

| No | Nama request | Metode | Alamat (pakai {{base_url}}) | Folder |
|---|---|---|---|---|
| 4 | Tambah Tanaman Baru | POST | {{base_url}}/plants | CRUD Tanaman |
| 5 | Ambil Satu Tanaman | GET | {{base_url}}/plants/{{plantId}} | CRUD Tanaman |
| 6 | Ubah Data Tanaman | PUT | {{base_url}}/plants/{{plantId}} | CRUD Tanaman |
| 7 | Hapus Tanaman | DELETE | {{base_url}}/plants/{{plantId}} | CRUD Tanaman |
| 8 | Ambil Tanaman ID Tidak Ada (404) | GET | {{base_url}}/plants/9999 | Skenario Gagal |
| 9 | Tambah Tanaman Tanpa Nama (400) | POST | {{base_url}}/plants | Skenario Gagal |
| 10 | Ambil Tanaman yang Sudah Dihapus (404) | GET | {{base_url}}/plants/{{plantId}} | Skenario Gagal |

# Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

## 3.1 Rekap assertion per request

| No | Nama request | Assertion status code | Assertion struktur atau isi JSON |
|---|---|---|---|
| 1 | Info API | Status 200 | Properti nama, nim, topik ada dan endpoints berupa array berisi 6 item |
| 2 | Ambil Semua Tanaman | Status 200 | Response array; tiap item punya 6 field bertipe benar; data id 1 adalah Kangkung |
| 3 | Filter Tanaman Jenis Sayur | Status 200 | Response array tidak kosong dan semua item berjenis sayur |
| 4 | Tambah Tanaman Baru | Status 201 Created | status "success", data memiliki 6 field, isi sama dengan body; id disimpan ke plantId |
| 5 | Ambil Satu Tanaman | Status 200 | Objek dengan 6 field; id sama dengan plantId dan namaTanaman "Tomat" |
| 6 | Ubah Data Tanaman | Status 200 | status "success", pesan berisi id, data berubah (Tomat Cherry, Bedeng 4, sudahPanen true) |
| 7 | Hapus Tanaman | Status 200 | status "success", pesan hapus berisi id, data bernilai null |
| 8 | Ambil Tanaman ID Tidak Ada (404) | Status 404 | status "error", pesan id 9999 tidak ditemukan, data null |
| 9 | Tambah Tanaman Tanpa Nama (400) | Status 400 | status "error", pesan "Field namaTanaman wajib diisi", data null |
| 10 | Ambil Tanaman yang Sudah Dihapus (404) | Status 404 | status "error", pesan id yang sudah dihapus tidak ditemukan, data null |

## 3.2 Cuplikan 1: request GET

Tempel test script **satu request GET** pilihan Anda.

```
// Test script request GET: Ambil Semua Tanaman

pm.test("Status code adalah 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Response berupa array dan tidak kosong", function () {
    const json = pm.response.json();
    pm.expect(json).to.be.an("array");
    pm.expect(json.length).to.be.at.least(3);
});

pm.test("Setiap tanaman memiliki field dengan tipe data yang benar", function () {
    const json = pm.response.json();
    json.forEach(function (plant) {
        pm.expect(plant).to.have.all.keys("id", "namaTanaman", "jenis", "lokasiBedeng",
"tanggalTanam", "sudahPanen");
        pm.expect(plant.id).to.be.a("number");
        pm.expect(plant.namaTanaman).to.be.a("string");
        pm.expect(plant.jenis).to.be.oneOf(["sayur", "buah", "hias"]);
        pm.expect(plant.tanggalTanam).to.match(/^\d{4}-\d{2}-\d{2}$/);
        pm.expect(plant.sudahPanen).to.be.a("boolean");
    });
});

pm.test("Data awal Kangkung dengan id 1 ada di dalam daftar", function () {
    const kangkung = pm.response.json().find(function (p) { return p.id === 1; });
    pm.expect(kangkung).to.not.be.undefined;
    pm.expect(kangkung.namaTanaman).to.eql("Kangkung");
});
```

**Penjelasan:**

Test pertama memeriksa status code 200 untuk memastikan server menjawab sukses. Test kedua memastikan response berupa array dan berisi minimal 3 data awal, sehingga bukan respons kosong atau objek. Test ketiga memeriksa setiap tanaman: memiliki tepat 6 field, id bertipe number, jenis hanya sayur/buah/hias, tanggalTanam berformat YYYY-MM-DD, dan sudahPanen bertipe boolean; ini mendeteksi perubahan nama atau tipe field. Test keempat memeriksa isi data, yaitu id 1 harus Kangkung, agar test tidak lulus hanya karena status 200.

## 3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
// Test script request POST: Tambah Tanaman Baru

pm.test("Status code adalah 201 Created", function () {
    pm.response.to.have.status(201);
});

pm.test("Response berstatus success dan berisi data tanaman baru", function () {
    const json = pm.response.json();
    pm.expect(json.status).to.eql("success");
    pm.expect(json.message).to.eql("Data tanaman berhasil ditambahkan");
    pm.expect(json.data).to.have.all.keys("id", "namaTanaman", "jenis", "lokasiBedeng",
"tanggalTanam", "sudahPanen");
    pm.expect(json.data.id).to.be.a("number");
});

pm.test("Isi data sama dengan yang dikirim pada body", function () {
    const data = pm.response.json().data;
    pm.expect(data.namaTanaman).to.eql("Tomat");
    pm.expect(data.jenis).to.eql("buah");
    pm.expect(data.lokasiBedeng).to.eql("Bedeng 2");
    pm.expect(data.tanggalTanam).to.eql("2026-09-25");
    pm.expect(data.sudahPanen).to.eql(false);
});

// Simpan id hanya jika POST berhasil, agar request berikutnya memakainya
if (pm.response.code === 201) {
    pm.environment.set("plantId", pm.response.json().data.id);
}
```

**Penjelasan:**

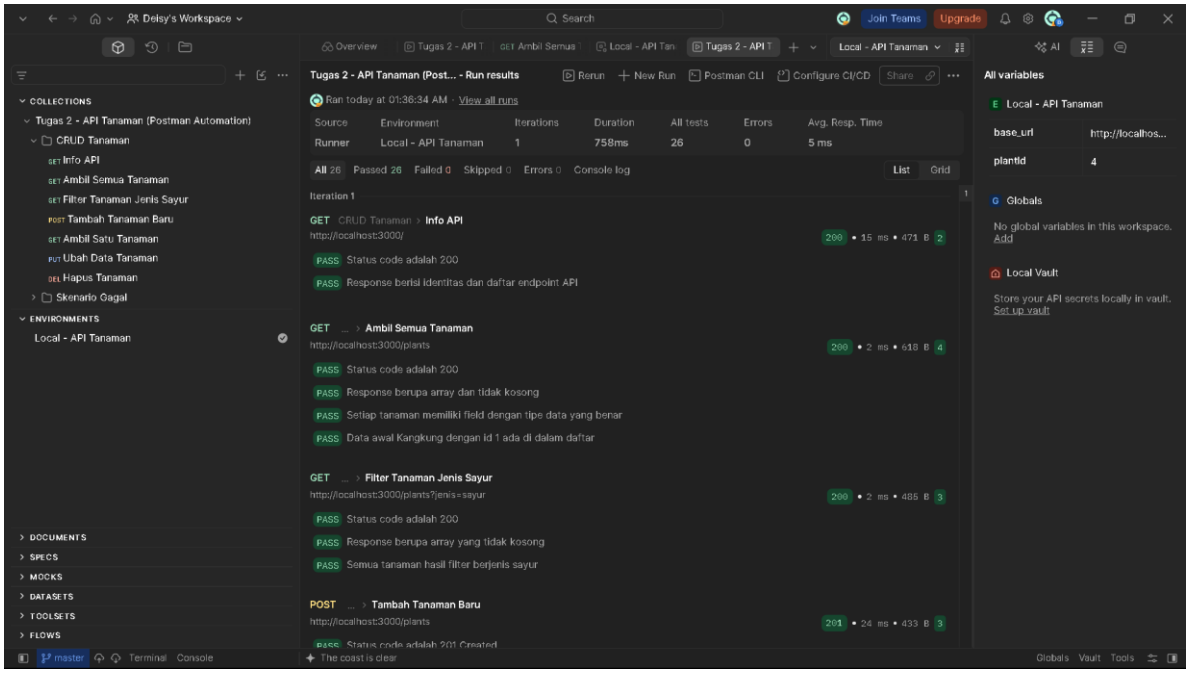
Test memeriksa status 201, struktur response (status "success", pesan, dan 6 field pada data), lalu isi data yang harus sama dengan body yang dikirim. Setelah itu, hanya bila status 201, script menyimpan data.id ke variabel Environment plantId dengan pm.environment.set. Request Ambil Satu Tanaman, Ubah Data Tanaman, Hapus Tanaman, dan Ambil Tanaman yang Sudah Dihapus memakai {{plantId}} pada alamatnya. Bila urutan ditukar sehingga POST berjalan belakangan, plantId masih kosong sehingga alamat menjadi /plants/ dan test pada request tersebut gagal.

# Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

## 4.1 Ringkasan Collection Runner

**Harus terlihat pada screenshot:** nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.




*Gambar 2. Ringkasan hasil Collection Runner*

**Penjelasan:**

*Jelaskan hasilnya: berapa request dan test yang dijalankan, apakah semuanya lulus, dan apa yang Anda lakukan sebelum menjalankan Runner (misalnya menyalakan ulang server).*

## 4.2 Rekap angka hasil

| | |
|---|---|
| **Nama Collection** | Tugas 2 - API Tanaman (Postman Automation) |
| **Environment yang dipakai** | Local - API Tanaman |
| **Jumlah request dijalankan** | 10 request |
| **Jumlah test lulus** | |
| **Jumlah test gagal** | |
| **Tanggal dan jam pengujian** | |

## 4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

| No | Request | Kegagalan (pesan) | Penyebab | Perbaikan |
|---|---|---|---|---|
| 1 | | | | |
| 2 | | | | |

# Bab 5 Newman di Terminal (Pengayaan)

> **Bagian ini opsional**
> Bab ini adalah pengayaan dan tidak mengubah kriteria penilaian. Isi bila Anda menjalankan Collection dengan Newman. Bila tidak, tulis "tidak dikerjakan" di bawah judul ini.

## 5.1 Hasil Newman

**Harus terlihat pada screenshot:** perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.

**[ Tempel screenshot di sini ]**

*Gambar 3. Hasil menjalankan Collection dengan Newman*

*Gambar 3. Hasil menjalankan Collection dengan Newman*

**Penjelasan:**

*Tulis perintah yang Anda jalankan dan jelaskan arti exit code (0 atau 1) pada hasil Anda.*

# Bab 6 Repositori dan Berkas

| | |
|---|---|
| **Tautan repositori** | https://github.com/DeisyCC32/tugas1-restful--2428240054- |
| **Lokasi berkas Collection (.json)** | postman/Tugas2_API_Tanaman.postman_collection.json |
| **Lokasi berkas Environment (.json)** | postman/Local_API_Tanaman.postman_environment.json |
| **Berkas Environment tidak memuat kunci rahasia sungguhan** | [X] Ya [ ] Tidak |
| **Repositori dapat dibuka dosen** | [ ] Ya [ ] Tidak |

# Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

## Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

**Penjelasan:**

*Tulis jawaban Anda di sini.*

## Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

**Penjelasan:**

*Tulis jawaban Anda di sini.*

# Lampiran A  Daftar Periksa Sebelum Mengumpulkan

| No | Butir yang diperiksa | Ya |
|---|---|---|
| 1 | Seluruh tanda `[...]` sudah diganti dengan data API saya | |
| 2 | Setiap endpoint API saya ada di Collection | |
| 3 | Alamat memakai `{{base_url}}`, bukan ditulis langsung | |
| 4 | Setiap request punya assertion status code dan struktur JSON | |
| 5 | Nama setiap test berupa kalimat yang menyatakan harapan | |
| 6 | POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya | |
| 7 | Server dinyalakan ulang sebelum Runner dijalankan | |
| 8 | Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas | |
| 9 | Berkas Environment tidak memuat kunci rahasia sungguhan | |
| 10 | Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS | |

# Lampiran B  Kriteria Penilaian

*Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.*

| Kriteria | Deskripsi | Dinilai dari |
|---|---|---|
| Kelengkapan Postman Collection | Seluruh endpoint project akhir tercakup dalam satu Collection | Bab 1 dan Bab 2 |
| Ketepatan automated test script | Assertion status code dan struktur response JSON tepat sasaran | Bab 3 |
| Keberhasilan Collection Runner | Seluruh test lolos saat Collection dijalankan | Bab 4 |
| Kejelasan dokumentasi hasil | Tangkapan layar ringkasan pengujian disertakan dengan jelas | Bab 4 dan seluruh dokumen |